Choosing a Fast Stand-In for a Slow Simulation: a Selection Tool and a 44-Problem Study

[Author names to be supplied by the authors][a]

[a] *[Affiliations and corresponding-author e-mail to be supplied by the authors]*

Abstract

STUB

Keywords: surrogate model, Gaussian process, neural network, reduced-order model, parametric PDE, model selection, learning curve

1. Introduction

Many engineering decisions rest on a simulation that is too slow to run as often as one would like. Choosing the shape of a part, finding how uncertain an input leaves the answer, or fitting a model to measurements can each need thousands of runs. A common remedy is a *surrogate*, or stand-in model: the simulation is run a limited number of times, say a few hundred, a cheap model is trained on the inputs and outputs, and the cheap model then replaces the simulation in all further work. (As an illustration, not a result of this paper: if one run takes an hour, five hundred runs take three weeks on one machine, whereas a trained stand-in answers in milliseconds.)

The practical question is which kind of stand-in to build, and how many runs are enough. One long-standing route compresses the output fields with proper orthogonal decomposition (POD) and regresses the weights of the leading modes on the inputs, with a neural network [1, 2] or a Gaussian process [3]. The other route trains a neural network to produce the whole field [4, 5]. Published comparisons are mixed. A survey of machine-learning solvers for fluid problems found that most papers claiming to beat a standard method used a weak baseline [6]; kernel and Gaussian-process methods have been reported to be competitive with DeepONet [7] and the Fourier neural operator [8] on benchmark problems [9]; and no universal winner was found between neural and polynomial surrogates [10]. A practitioner holding a few hundred runs of their own simulation still has no procedure that tells them

which stand-in to use on *their* data, how far it can be trusted, or how many more runs it would take to reach a target accuracy.

Gap.. Existing comparisons rank methods on fixed problem sets and do not give the practitioner a validated procedure to select a stand-in, to see whether the POD basis limits accuracy, and to forecast the number of runs needed.

Motivation and contributions

1. **A selection tool, SurroSelect** (section 4). It fits a pool of stand-in models, chooses by cross-validation while reporting which models are tied with the best, checks whether the POD basis is the bottleneck, and forecasts the error for more runs from a learning curve. It is free software with a command-line interface (section S7supplement.pdf).
2. **A pre-registered evaluation on independent problems** (section 6.3). Seven criteria were fixed before any model was fitted to eight problems solved by the third-party finite-element library `scikit-fem`; we report which were met and which were not, against a neural network tuned with twice the budget of any other method.
3. **A 44-problem study of where each family wins and why** (section 6.1). Controls separate the effects of the reduced basis, of the regressor, and of how widely its hyper-parameters are searched; the outcome of a comparison is shown to depend on training budget, tuning effort and basis rank.
4. **An honest record of what did not work** (section 7), including a pilot-data diagnostic that was not confirmed, a control that had to be widened after review, and every deviation from the plan (Supplement, section S4supplement.pdf).

The paper proposes no new regression model; the pool consists of classical methods, and the contribution is the procedure, its validation, and the evidence about when each member wins.

Organisation

Section 2 reviews related work; section 3 fixes notation and assumptions; section 4 describes the method; section 5 the experimental framework; section 6 the results; section 7 the discussion and limitations; section 8 concludes.

2. Related Work

2.1. Reduced-basis surrogates with learned coefficients

Non-intrusive reduced-order models regress the weights of POD modes on the parameters. Hesthaven and Ubbiali used neural networks for this [1], Bhattacharya et al. analysed PCA-based neural operators [2], and Guo and Hesthaven used Gaussian-process regression for nonlinear structural analysis [3]. Bērziņš et al. compared radial-basis-function, Gaussian-process and neural-network regression inside one POD framework for extruder flows and reported several advantages of the Gaussian process [11]; we read only the abstract, and it is the closest precedent for our finding on smooth problems. Mora et al. proposed a hybrid Gaussian-process/neural operator framework [12].

2.2. Neural networks, operators and nonlinear manifolds

DeepONet [7], the Fourier neural operator [8] and the neural-operator framework [13] learn maps between function spaces; autoencoders learn nonlinear latent manifolds for reduced models [14, 4, 5]. Our parameters are finite-dimensional, so we study direct and bottleneck networks rather than operator-input architectures. Whether a small linear basis can work depends on the decay of the Kolmogorov n-width [15], which is slow for wave problems with discontinuous data [16].

2.3. Kernel and Gaussian-process methods

Gaussian processes are the standard low-dimensional regressor [17]. Batlle et al. develop a kernel framework with a priori error analysis and report it competitive with DeepONet and the Fourier neural operator [9]. A position paper argues that many neural-surrogate successes sit on low-dimensional manifolds where any competent reduced model interpolates well [18].

2.4. Fair comparison, tuning and reporting

McGreivy and Hakim found that 79% of 76 reviewed articles compared with weak baselines [6]. De Hoop et al. study the cost–accuracy trade-off of neural architectures [19]; Westermann et al. compare neural and polynomial surrogates by Pareto frontiers and find no universal winner [10]; a Common Task Framework has been proposed to evaluate scientific machine-learning methods against curated baselines [20]. Random search is an efficient alternative to grid search [21]; adaptive early-stopping schemes such as Hyperband [22] and software such as Optuna [23] reduce tuning cost; reporting results as a function of tuning budget and accounting for run-to-run variance have been advocated [24, 25].

Table 1: Closest related work. A dash means that we did not find the property in the parts we read (mostly abstracts); it does not prove absence.

Work	Kernel/GP/poly vs neural compared	Tuning or baselines analysed	Ties reported	Selection guidance	Basis check, runs forecast	Open tool
Hesthaven & Ubbiali [1]	–	–	–	–	–	–
Guo & Hesthaven [3]	–	–	–	–	–	–
Bērziņš et al. [11]	✓	–	–	–	–	–
Batlle et al. [9]	✓	–	–	–	–	–
Westermann et al. [10]	✓	–	–	–	–	–
McGreivy & Hakim [6]	–	✓	–	–	–	–
Williams & Cremaschi [26]	–	–	–	✓	–	–
This paper	✓	✓	✓	✓	✓	✓

2.5. Surrogate selection, learning curves and software

Williams and Cremaschi compared eight surrogate forms on generated data to look for rules for choosing a form [26]. Learning curves, the error as a function of training-set size, have been surveyed for decision making in supervised learning [27], and error scaling laws have been fitted across data scales in deep learning [28]. General-purpose libraries for model-order reduction [29] and uncertainty quantification [30] exist; we did not compare their features with ours, and the finite-element library `scikit-fem` [31] is used here only to generate independent test problems. PDEBench [32] provides large simulated benchmarks for scientific machine learning; we did not use it (section 7).

2.6. Comparison with prior work

Table 1 lists what each closest work contains, judged from the parts we read (mostly abstracts; Westermann et al. in large part).

2.7. Motivations

(i) Method comparisons are reported on fixed problem sets, so a practitioner cannot tell whether a published ranking applies to their problem. (ii) The outcome of a comparison depends on tuning effort [6, 24], so a tool must make the tuning effort explicit and equal. (iii) Small differences are often ties, so a tool should report ties instead of a single winner. (iv) The accuracy of any POD-based model is bounded by the basis (proposition 2), which should be checked before tuning a regressor. (v) The practical question is the number of runs needed, which learning-curve extrapolation can answer only roughly and must be validated, not assumed.

3. Preliminary Concepts

Problem.. A parametric simulation maps an input vector $\boldsymbol{\mu} \in [0,1]^p$ (rescaled to the unit box) to an output field $\mathbf{u}(\boldsymbol{\mu}) \in \mathbb{R}^m$ sampled on a fixed grid. We are given N runs $\{(\boldsymbol{\mu}_i, \mathbf{u}_i)\}_{i=1}^N$ and want a model $\hat{\mathbf{u}}$ that is accurate on new inputs. Accuracy is the mean relative L_2 error over n held-out inputs,

$$\mathrm{err}(\hat{\mathbf{u}}) = \frac{1}{n} \sum_{j=1}^{n} \frac{\|\hat{\mathbf{u}}(\boldsymbol{\mu}_j) - \mathbf{u}(\boldsymbol{\mu}_j)\|_2}{\|\mathbf{u}(\boldsymbol{\mu}_j)\|_2}. \tag{1}$$

POD basis.. With $\bar{\mathbf{u}}$ the mean of the training fields and $V_r \in \mathbb{R}^{m \times r}$ the r leading left singular vectors of the centred snapshot matrix, a POD model has the form

$$\hat{\mathbf{u}}(\boldsymbol{\mu}) = \bar{\mathbf{u}} + V_r \, \hat{\mathbf{a}}(\boldsymbol{\mu}), \tag{2}$$

where $\hat{\mathbf{a}}$ regresses the coefficients $\mathbf{a} = V_r^\top (\mathbf{u} - \bar{\mathbf{u}})$ on $\boldsymbol{\mu}$.

Regressors.. Gaussian process (GP): the posterior mean with a Matérn-5/2 or squared-exponential kernel and one length-scale per input (ARD), hyper-parameters fitted by maximising the marginal likelihood [17]. *Kernel ridge regression (KRR):* $\hat{a}(\boldsymbol{\mu}) = \sum_i \alpha_i k(\boldsymbol{\mu}, \boldsymbol{\mu}_i)$ with $\boldsymbol{\alpha} = (K + \lambda N I)^{-1}\mathbf{a}$. *Polynomial:* ridge regression on total-degree monomials.

Ridge hat matrix and leave-one-out.. For a ridge-type smoother the fitted values are $H\mathbf{a}$ with $H = K(K + \lambda N I)^{-1}$, and the leave-one-out residual of run i is $(a_i - \hat{a}_i)/(1 - H_{ii})$ (proposition 1).

Tie band.. Two errors are *tied* if their ratio lies within $10^{\pm 0.1} \approx 1.26$; the outcome of a comparison of groups A and B is $\bar{y} = \log_{10}(\mathrm{err}_B/\mathrm{err}_A)$, positive when A is better, and a cell is a tie if $|\bar{y}| \le 0.1$.

Assumptions..

A1 Inputs of training and test runs are drawn independently from the same distribution on the unit box; extrapolation tests (outside the box) are reported separately.

A2 The output grid is the same for every run and the solver is deterministic (no noise).

A3 Runs are exchangeable, so that k-fold cross-validation estimates the error on new runs. This fails for adaptive or clustered designs; the tool does not detect that.

A4 The number of inputs is moderate (up to a few tens in our tests; only five of our 44 problems have ten or more).

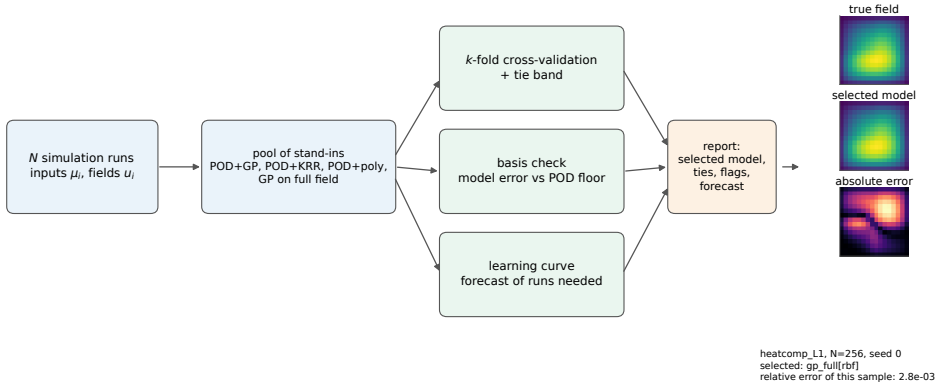


Figure 1: SurroSelect pipeline. The N simulation runs enter a pool of stand-in models; k-fold cross-validation selects the best model and reports which others are tied with it; the basis check compares the model error with the error of projecting held-out fields onto the POD basis; the learning curve forecasts the error for more runs. The example fields are taken from the first test sample of problem `heatcomp_L1` (seed 0, $N = 256$): true field, selected model, and absolute error, as chosen by the rule in section 6.5.

4. Proposed Methodology: SurroSelect

4.1. Overview

SurroSelect takes the N runs and returns (a) the selected stand-in, (b) the set of models tied with it, (c) a basis check, and (d) a learning-curve forecast (fig. 1, algorithm 1). Every step uses only the training runs.

4.2. Candidate pool

The default pool has seven members, all classical: POD+GP with a Matérn-5/2 kernel and rank 32, with a squared-exponential kernel and rank 32, and with Matérn-5/2 and rank 8 (GP on the four leading modes with marginal-likelihood fitting, remaining modes sharing the fourth kernel); POD+KRR-wide, whose kernel, length-scale (0.05–$15\sqrt{p}$ on the unit box) and ridge (10^{-16}–10^{-2} times N) are chosen over a wide grid by the leave-one-out error of proposition 1; POD+polynomial, degree up to six with at most 3000 features, degree and ridge chosen by leave-one-out; and a GP directly on the full field (Matérn-5/2 or squared-exponential, shared ARD kernel fitted on at most 300 runs), included only when $m \leq 2000$. A tuned neural network (`TunedMLP`, section 5) can be added as an optional member.

4.3. Cross-validated, tie-aware selection

Each member is scored by k-fold cross-validation ($k = 4$ by default) with all fitting, including hyper-parameter fitting, repeated inside each

Algorithm 1: SurroSelect (default settings)

Input: runs $(\boldsymbol{\mu}_i, \mathbf{u}_i)_{i \le N}$, folds k, tie band $\tau = 0.1$ decade, optional target error ε

Scale inputs to the unit box; split the runs into k folds;

foreach *pool member* c **do**

  fit c on each training fold and score it on the held-out fold; $e_c \leftarrow$ mean error (1);

$c^\star \leftarrow \arg\min_c e_c$; tied set $\leftarrow \{c : \ e_c \le 10^\tau e_{c^\star}\}$; refit $c^\star$ on all N runs;

Basis floor $\phi \leftarrow$ mean projection error of held-out fields onto a rank-32 POD basis; flag the basis if $e_{c^\dagger} \le 1.25\,\phi$ for the best POD-based $c^\dagger$;

Learning curve: errors of $c^\star$ at $n \in \{N/8, N/4, N/2, N\}$ (training-fold sizes); fit $\log e = \log A - b \log n$ on the last three points (and on all four); forecast $e(4N)$; if ε is given, return $(A/\varepsilon)^{1/b}$ unless $b < 0.1$ or the result exceeds $16N$;

return *$c^\star$, tied set, basis flag, forecast*

training fold. The member with the smallest mean cross-validated error (1) is selected, and all members within the tie band of section 3 are reported as tied. Members that fail to fit receive infinite error.

4.4. Basis check

On each fold, a rank-32 POD basis is fitted on the training part and the held-out fields are projected onto it; the mean relative error of this projection is the *basis floor*. If the best POD-based member's cross-validated error is within 25% of the floor, the tool flags the basis as limiting and recommends a full-field model, a larger basis, or a nonlinear output map. The reason is proposition 2.

4.5. Budget forecast

For the selected member, cross-validated errors are computed at 1/8, 1/4, 1/2 and all of the training folds. A power law $e(n) = A\,n^{-b}$ is fitted by least squares in log–log coordinates to the last three points (central forecast) and to all four (alternative); the lower and upper of the two forecasts form a range. The number of runs needed for a target error ε is $(A/\varepsilon)^{1/b}$; the tool declines to answer if $b < 0.1$ or if the answer exceeds sixteen times the current number of runs. This is an extrapolation and is validated in section 6.3, not assumed.

4.6. Theory

Two results are standard and are included because the tool relies on them; proofs are in the Supplement (section S1supplement.pdf).

Proposition 1 (Closed-form leave-one-out)**.** *For a ridge smoother with hat matrix H (so $\hat{\mathbf{a}} = H\mathbf{a}$ on the training inputs), the leave-one-out prediction error at run i equals $(a_i - \hat{a}_i)/(1 - H_{ii})$. For $H = K(K + \lambda N I)^{-1}$ with $K = Q\,\mathrm{diag}(w)Q^\top$, $1 - H_{ii} = \sum_j Q_{ij}^2\, \lambda N/(w_j + \lambda N)$, which can be evaluated without cancellation when λN is tiny.*

Proposition 2 (Basis floor)**.** *Let $\hat{\mathbf{u}}$ be any model whose outputs lie in the affine subspace $\bar{\mathbf{u}}+\mathrm{span}(V_r)$. Then for every run, $\|\hat{\mathbf{u}}-\mathbf{u}\|_2 \geq \|(I-V_rV_r^\top)(\mathbf{u}-\bar{\mathbf{u}})\|_2$, so the mean relative error of $\hat{\mathbf{u}}$ is at least the mean relative projection error of the basis, whatever the regressor.*

The first is the PRESS identity for ridge regression [33]; the second is the optimality of orthogonal projection. The forecast has no guarantee: it is exact only if the error follows a power law, and section 6.3 tests how often it is useful.

5. Experimental Framework

5.1. Setup

Data protocol.. For each problem we generate 4096 candidate training inputs, 1000 held-out test inputs and 500 extrapolation inputs (coordinates drawn from $[-0.25, 1.25]^p$, rejected if all lie inside the unit box) from streams seeded by the problem name; solutions are computed once and cached. A seed $s \in \{0, 1, 2\}$ fixes a permutation of the pool, and the training set for budget N is its first N elements, so training sets are nested within a seed. Budgets are $N \in \{32, \ldots, 1024\}$ for 24 problems and $\{64, 256, 1024\}$ for the others.

Problems.. Table 2 lists the 44 problems. 36 come from our own solvers, each verified against exact or manufactured solutions or by grid refinement; they are grouped into 6 classes named after application areas (the classes are our own groupings of standard PDE families; nothing was calibrated to a real application). 8 problems were added in the final stage and solved with the independent finite-element library `scikit-fem` (version 12.0.2); their unit tests compare with a series solution, a scaling law, load reversal, and mesh and time-step convergence. All data are simulated.

Table 2: The 44 test problems by class. "Inputs" is the number of parameters p, "Outputs" the number of field values m.

Class	Problems	Inputs	Outputs	Solver
Composite heat conduction	2	4–9	256	scikit-fem (third party)
Convection–diffusion	2	3–4	256	scikit-fem (third party)
Elastic L-shaped plate	2	4–7	130	scikit-fem (third party)
Transient heat, moving source	2	3–5	192	scikit-fem (third party)
Flow through porous rock	9	3–32	256	our code
Heat conduction	9	3–8	128–256	our code
Pollutant or heat carried by a flow	4	4	128–256	our code
Spreading fronts and reactions	4	3–4	128–256	our code
Thermal runaway	2	3	256	our code
Waves, shocks, acoustics	8	3–4	128–256	our code

Hardware and software.. All runs used 4 CPU cores (Intel Xeon @ 2.10GHz), no GPU, Python 3.13.16, NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.1, PyTorch 2.14.1 and Optuna 5.0.0; each model fit used one thread.

5.2. Metrics

The error is (1). The comparison between a group A (here the reduced-basis group) and a group B (the direct networks) is

$$\bar{y} = \frac{1}{S} \sum_{s=1}^{S} \log_{10} \frac{\mathrm{err}_B^{(s)}}{\mathrm{err}_A^{(s)}}, \tag{3}$$

over seeds, with the best member of each group chosen on validation data; $\bar{y} > 0.1$ means A is clearly better, $|\bar{y}| \le 0.1$ is a tie. For the tool, the *regret* of a cell is $\mathrm{err}(\text{selected}) / \min_{c \in \text{pool}} \mathrm{err}(c)$, with all errors measured on the 1000 test inputs; the *forecast ratio* of a case is $\hat{e}(4N_0)/e_{\text{actual}}(4N_0)$, where the actual error is that of the same pool member refit on $4N_0$ runs. Cell-wise values are geometric means over the three seeds.

5.3. Baselines

- *Five benchmark methods* under one tuning protocol: POD+GP, POD+polynomial, POD+MLP (reduced-basis group) and a direct MLP and a bottleneck MLP (network group). Each gets eight random-search trials (the first is a fixed default) scored on a random fifth of the training runs; networks use Adam for 1500 epochs in single precision.

- *Always POD+GP*: the benchmark's POD+GP, i.e. no selection.

Table 3: Pre-registered hypotheses for the tool and their pass criteria (cells: 8 problems $\times$ $N \in \{64, 256, 1024\} = 24$).

ID	Statement	Pass criterion
H1a	Cross-validated selection is close to the oracle	median regret ≤ 1.5 and $\geq 75\%$ of cells ≤ 2
H1b	Not worse than always using POD+GP	median ratio ≤ 1
H1c	At least as good as the benchmark's own selection	median ratio ≤ 1; more cells better than worse by $> 10\%$
H2	The forecast is usable	$4N_0$ forecast within a factor 2 in $\geq 70\%$ of 48 cases; reach-target classification $\geq 75\%$
H3	The basis flag identifies cases where a full-field model is better	$\geq 75\%$ of flagged cells (if at least five)
H4	A strongly tuned network rarely beats the tool	better by $> 25\%$ in at most 4 of 16 cells
H5	The benchmark finding holds on independent problems	at least 5 of 8 problems reduced-basis-clear at $N = 1024$

- *Oracle*: the best pool member by test error (an upper bound; unavailable in practice).
- *Strongly tuned network*: `TunedMLP`: Optuna TPE with successive-halving pruning [23, 22], 16 trials over output type (full field or POD rank 8/16/32), depth 2–4, width 64–256, activation, learning rate and weight decay; full-batch float64 training; final refit on all runs with four times the epochs and 200 L-BFGS iterations. It receives twice the tuning budget of any other method and is selected on a validation split of the training runs only.

Controls for the mechanism study are described in section 6.1.

5.4. Hypotheses and how each is tested

The hypotheses, metrics and pass/fail criteria below were committed to version control before the corresponding results (commit hashes in Supplement, section S2supplement.pdf); table 3 lists them. Earlier-stage hypotheses (pilot diagnostic, mechanism controls, longer training) are in the Supplement (section S3supplement.pdf). Binomial proportions are given with Wilson 95% intervals [34]; medians with percentile bootstrap intervals (2000 resamples over cells); the tests against the thresholds are one-sided exact binomial tests with Holm adjustment [35] over the three that admit one.

6. Results and Analysis

All numbers in this section are written by `code/experiments/aggregate.py` from the result files. Cell-wise values are geometric means over three seeds; intervals are 95%.

6.1. Where each family wins, and by how much (the evidence base)

Main problems.. Figure 2 shows the outcome $\bar{y}$ of (3) for the reduced-basis group against the direct networks. Among the 24 main problems (those with the full budget range), the reduced-basis group is clearly better in 5 problems at $N = 32$, the networks in 5, and 14 are ties; at $N = 1024$ the counts are 13, 8 and 3. The number of network wins at $N = 1024$ is 6 if the tie band is widened to 0.15 decade and 10 if it is narrowed to 0.05, so counts of ties and small wins are not stable. Where the reduced basis wins, it can win by orders of magnitude (panel b, top rows); where networks win, the largest margin is a factor of about three.

Accuracy reached.. For a coarse target of 10% error, both groups reach it on all 36 own-solver problems within 1024 runs; for 1% they are about equal (27 against 28 problems); at the demanding end, 0.1%, the reduced-basis group reaches it on 19 problems and the networks on 10 (9 against 4 among the 24 main problems, excluding the eight smooth problems we added later because we expected the Gaussian process to do well). On our own problems a solve takes a median of 6.0 ms, so fitting a surrogate costs more than running the simulation 1024 times on most of them; the case for surrogates rests on simulations much slower than ours, and the comparison that matters is accuracy per run.

Independent problems (H5).. On the eight problems solved by `scikit-fem`, the reduced-basis group is clearly better at $N = 1024$ in 3 of 8; the criterion of at least five (H5) is therefore **not met**. The other problems are ties or network wins (table S5supplement.pdf): the networks are ahead on the composite-conduction problem with nine inputs and high contrast and on both moving-source problems. The simple picture "reduced basis wins at large budgets" therefore does not carry over to three of these eight independent problems; the tool of section 4 is designed for exactly this case, and section 6.3 tests it.

6.2. Why the Gaussian-process reduced model wins when it does

Figure 3 compares alternatives with the Gaussian-process reduced model in the 28 budget–problem combinations (budgets 64, 256, 1024) where it

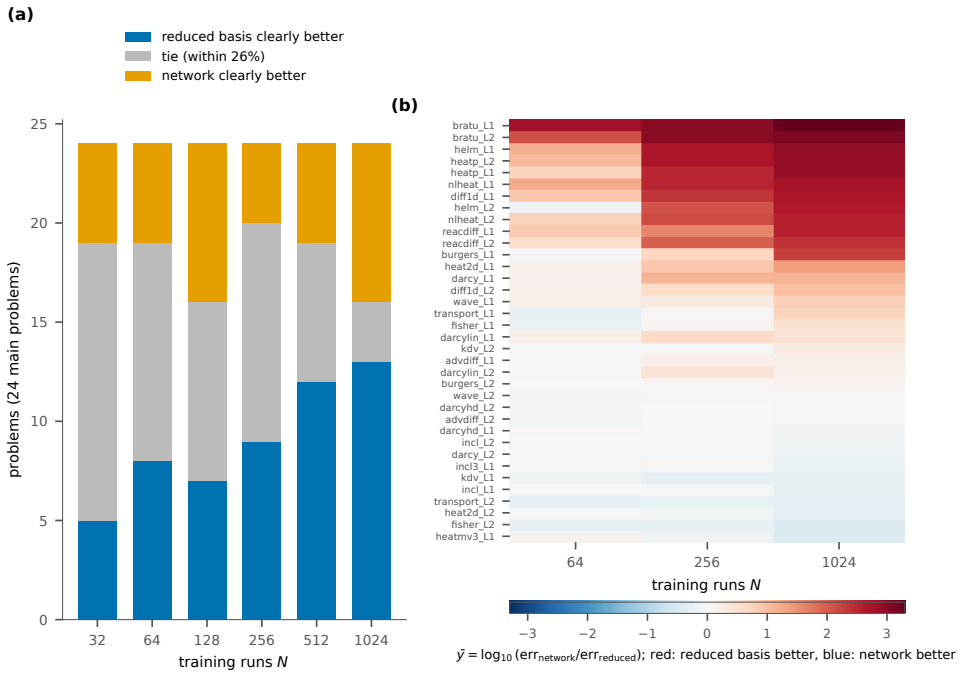


Figure 2: The reduced-basis group wins clearly on smooth problems at large budgets, and most differences at small budgets are ties. (a) Number of the 24 main problems in which the reduced-basis group is clearly better, tied, or clearly worse than the direct networks, by training budget (simulated benchmark problems, five methods, three seeds, tie band ±0.1 decade). (b) Signed effect $\bar{y}$ for every problem with budgets 64, 256 and 1024; the colour scale is centred at zero. The $N = 1024$ results are confounded with a doubling of the networks' optimiser steps (section S3supplement.pdf).

clearly beats the best network (the comparison uses the best network by *test* error, an oracle that favours the networks). Kernel ridge with the narrow range of ridge and length-scale of our first control is a median 13 times less accurate; with a wide range and the same eight tuning trials, 6.5 times; with 32 trials, 2.7 times (57% of cells within a factor three), and the wide kernel ridge beats the best network clearly in 64% of these cells. A Gaussian process on the full field without POD is as accurate as the POD-based one (median ratio 0.95). The Gaussian-process advantage is therefore largely an advantage of a smooth regressor whose regularisation is searched widely and finely, and not of the reduced basis; we did not isolate the contribution of marginal-likelihood fitting, because its cost is not matched by the kernel-ridge search. The wide-range control was added after an AI-agent review of an earlier draft showed that our first control could not reach the

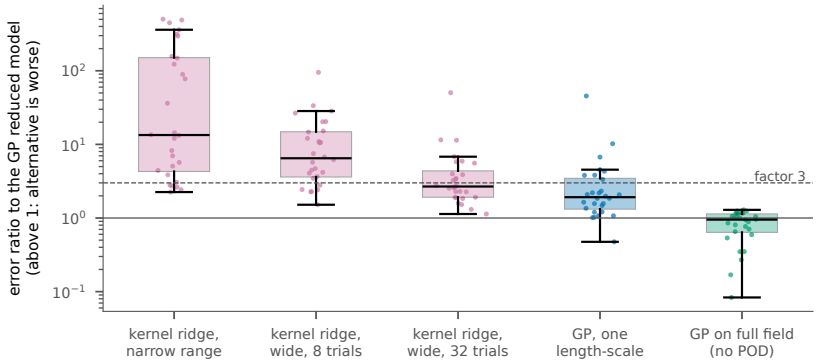


Figure 3: Widening the hyper-parameter search of kernel ridge closes most of the gap to the Gaussian-process reduced model, and a Gaussian process on the full field is as good as the POD-based one. Error ratio of each alternative to the reduced-basis Gaussian process in the 28 budget–problem combinations where that model clearly beats the best network (own-solver problems, budgets 64, 256, 1024); boxes show quartiles, dots individual combinations, the dashed line a factor of three.

regularisation that near-interpolating fits need; this is a post hoc control (Supplement, section S2supplement.pdf). Three further results qualify the picture. Training the networks four times longer moved the verdict in 15 of 48 cases, always toward the networks, and improved 90% of the refits; the large gaps survived stronger training (double precision, 6000 steps, L-BFGS) in 5 of 7 decisive problems, with the networks' error floor lowered by a median 0.47 decade to about 2.0×10^{-4}. On the extrapolation test set the median $\bar{y}$ at $N = 1024$ is -0.03 instead of 0.16. And the class of a cell agrees between halves of the random-search draws in 83% of cases (sign of clear differences: 99%), so small differences should not be over-interpreted. The earlier-stage hypotheses, including the pilot diagnostic that was not confirmed, are in the Supplement (section S3supplement.pdf).

6.3. Evaluation of SurroSelect on independent problems

The tool was run on the eight `scikit-fem` problems at $N \in \{64, 256, 1024\}$ with three seeds (24 cells), with every pool member also refit and scored on the 1000 test inputs so that the oracle choice is known. Table 4 lists the pre-registered criteria and what was observed; table 5 gives the errors per problem.

Selection quality (H1a, H1b, H1c).. Cross-validated selection chose the oracle member in most cells, and the regret was at most two in all 24 cells: median regret 1.00 (interval 1.00–1.02), 100% of cells within a factor two (Wilson

Table 4: Pre-registered criteria (table 3) and results on the eight independent problems. Intervals are 95% (bootstrap for medians, Wilson for proportions); the Holm-adjusted p-values are one-sided exact binomial tests against the thresholds.

ID	Criterion	Observed (95% interval)	Holm p	Verdict
H5	reduced basis clear on ≥ 5 of 8	3 of 8	–	not met
H1a	median regret ≤ 1.5; $\geq 75\%$ of cells ≤ 2	median 1.00 [1.00, 1.02]; 100% [86%, 100%]	0.003	met
H1b	median ratio vs always-GP ≤ 1	0.85 [0.59, 0.97]; better/worse 15/1	–	met
H1c	median ratio vs benchmark selection ≤ 1; better $>$ worse	1.01 [0.59, 1.19]; better/worse 11/9	–	not met
H2	forecast within $\times 2$ in $\geq 70\%$ of 48	37 of 48 (77% [63%, 87%])	0.363	met
H2 (reach 1%)	classification accuracy $\geq 75\%$	100%	–	met
H2 (reach 0.1%)	classification accuracy $\geq 75\%$	88%	–	met
H3	$\geq 75\%$ of flagged cells (if ≥ 5)	15 flagged; 15 better (100%)	–	met
H4	strong network better by $> 25\%$ in ≤ 4 of 16	4 of 16	0.630	met

interval 86%–100%; H1a met). Choosing a member of the pool at random would give a median regret of 3.77 and put only 38% of the cells within a factor two, so the selection is far from chance. The tool was no worse than always using the benchmark's POD+GP (median ratio 0.85, interval 0.59–0.97; better by more than 10% in 15 cells, worse in 1; H1b met). Against the benchmark's own selection among the five methods, which includes neural networks, the median ratio was 1.01 (interval 0.59–1.19); the tool was better by more than 10% in 11 cells and worse in 9, so H1c is **not met**. The cells where the tool loses (largest ratio 2.2 on `heatcomp_L2` at $N = 1024$; 1.4 on `lshape_L2` at $N = 1024$) are those in which a network from the benchmark beats every member of the default pool; the default pool contains no network.

Budget forecast (H2).. For the 48 cases, the forecast from N_0 runs of the error at $4N_0$ runs was within a factor two of the actual error in 37 (77%, Wilson interval 63%–87%; met), but the interval includes the threshold of 70% (one-sided exact binomial $p = 0.18$ against it, Holm-adjusted 0.363). The result depends strongly on the starting budget: 92% of the cases from $N_0 = 64$ but only 15 of 24 (62%) from $N_0 = 256$, where 7 forecasts were too low by more than a factor two (fig. 4); overall 7 forecasts were too low and 4 too high. Errors that fall faster than a power law over the last data points are extrapolated too optimistically. The range between the two fits contains the actual error in only 12% of the cases and must not be read as an uncertainty interval. For the decision "will 1% (0.1%) be reached at 1024 runs?" from 256 runs, the forecast was right in 100% (88%) of 24 cases;

Table 5: Test error (geometric mean over three seeds) on the eight independent problems: SurroSelect, the oracle (best pool member), always POD+GP, the benchmark's validation-selected method, and the strongly tuned network.

Problem	N	Tool	Oracle	Always GP	Benchmark sel.	Tuned net
convdiff_L1	256	4.2×10^{-4}	4.2×10^{-4}	1.6×10^{-3}	1.6×10^{-3}	2.4×10^{-3}
convdiff_L2	256	1.1×10^{-2}	1.1×10^{-2}	1.5×10^{-2}	1.5×10^{-2}	1.1×10^{-2}
heatcomp_L1	256	2.2×10^{-3}	2.0×10^{-3}	2.0×10^{-3}	2.0×10^{-3}	3.7×10^{-3}
heatcomp_L2	256	1.6×10^{-1}	1.6×10^{-1}	1.9×10^{-1}	1.0×10^{-1}	1.2×10^{-1}
heatmove_L1	256	9.0×10^{-4}	9.0×10^{-4}	7.5×10^{-3}	7.5×10^{-3}	5.5×10^{-3}
heatmove_L2	256	2.1×10^{-2}	2.1×10^{-2}	3.2×10^{-2}	3.2×10^{-2}	2.7×10^{-2}
lshape_L1	256	8.5×10^{-4}	8.5×10^{-4}	8.5×10^{-4}	8.5×10^{-4}	6.2×10^{-3}
lshape_L2	256	7.6×10^{-2}	7.6×10^{-2}	7.4×10^{-2}	5.3×10^{-2}	3.7×10^{-2}
convdiff_L1	1024	1.4×10^{-5}	1.4×10^{-5}	1.0×10^{-3}	1.0×10^{-3}	2.1×10^{-3}
convdiff_L2	1024	1.9×10^{-3}	1.8×10^{-3}	8.3×10^{-3}	8.3×10^{-3}	5.6×10^{-3}
heatcomp_L1	1024	1.2×10^{-4}	1.2×10^{-4}	4.3×10^{-4}	4.3×10^{-4}	2.1×10^{-3}
heatcomp_L2	1024	8.1×10^{-2}	8.0×10^{-2}	9.0×10^{-2}	3.7×10^{-2}	3.6×10^{-2}
heatmove_L1	1024	9.1×10^{-5}	9.1×10^{-5}	6.6×10^{-3}	4.5×10^{-3}	3.5×10^{-3}
heatmove_L2	1024	4.2×10^{-3}	4.2×10^{-3}	2.1×10^{-2}	1.5×10^{-2}	1.3×10^{-2}
lshape_L1	1024	7.1×10^{-5}	6.1×10^{-5}	1.2×10^{-4}	1.2×10^{-4}	3.6×10^{-3}
lshape_L2	1024	2.8×10^{-2}	2.8×10^{-2}	2.5×10^{-2}	2.0×10^{-2}	1.7×10^{-2}

always guessing the more common answer would be right in 75% (50%) of them.

Basis check (H3).. The basis flag was raised in 15 cells, belonging to the convection–diffusion and transient-heat problems; in 15 of them (100%) a full-field Gaussian process had a test error more than 10% below the best POD-based member (H3 met). The flagged problems are those where the benchmark found networks competitive with the POD-based Gaussian process, which is consistent with proposition 2.

Against a strongly tuned network (H4).. The network tuned with twice the budget of any other method was better than the tool-selected model by more than 25% in 4 of 16 cells (H4 met); the median ratio of the tool's error to the tuned network's was 0.34, the tool was better by more than 10% in 11 cells and the network in 4. The tuned network needed a median of 146 s ($N = 256$) and 429 s ($N = 1024$) on one core, against 112 and 312 s for the tool's cross-validated selection (section 6.4).

6.4. Ablation, noise and cost

Ablation.. Restricting the pool, with selection by the stored cross-validation errors (table 6), shows which members matter: without the full-field Gaussian process the regret is within a factor two in only 75% of cells, and without any Gaussian-process member (ridge and polynomial only) the test error is a median 2.7 times larger than with the full pool.

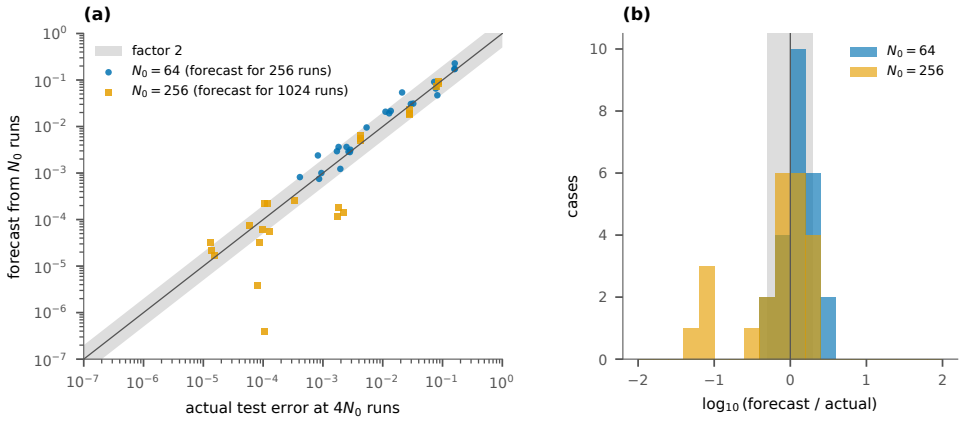


Figure 4: The learning-curve forecast is useful from small budgets and too optimistic from larger ones. (a) Forecast against actual test error at four times the starting budget (48 cases; independent problems; shaded: within a factor two). (b) Histogram of $\log_{10}$(forecast/actual); the cases below -0.3 are forecasts that are too low.

Table 6: Ablation of the pool on the eight independent problems (24 cells); regret as in section 5.

Pool	Median regret	Cells with regret ≤ 2	Median error vs full pool
full pool	1.00	100%	1.00×
POD-based members only	1.07	75%	1.02×
no full-field GP	1.07	75%	1.02×
GP members only	1.01	96%	1.00×
no GP (ridge, polynomial)	2.81	33%	2.74×

Sampling noise.. A pool member picked at random gives median regret 3.77, and the seed-to-seed spread of the tool's regret is small (median regret 1.01, 1.00 and 1.00 at the three budgets), so the observed regrets are not explained by chance. Seeds share the pool and test set and nest their training subsets, so seed variability understates the variability of a new sample of runs.

Computational cost.. On 4 cores (Intel Xeon @ 2.10GHz), one thread per fit, selection with cross-validation took a median of 7, 112 and 312 s at $N = 64$, 256 and 1024; adding the learning curve brought this to 9, 134 and 426 s. The solvers of the independent problems take 3–27 ms per run, so for these problems the tool costs more than running the simulation 1024 times; its use is justified when one run takes minutes or hours.

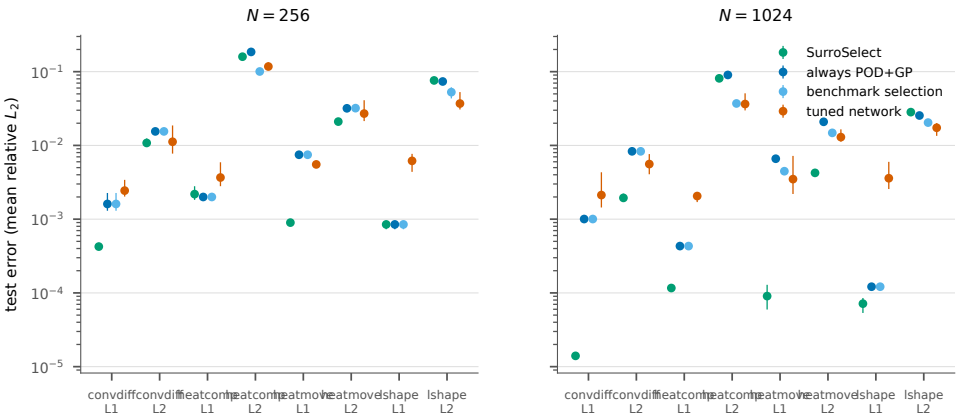


Figure 5: Test error of the tool and its competitors on each independent problem. Points: geometric mean over three seeds; bars: minimum to maximum over seeds.

6.5. Qualitative analysis

Figure 6 shows the first test sample in dataset order for the first problem of each family at $N = 256$ (seed 0), and two failure cases selected by rules fixed before the figure was made: the problem with the largest selection regret at $N = 256$ and the problem with the largest test error. The selected model reproduces the large-scale structure of the fields in the four rule-selected examples, with relative errors given beside each panel; the error maps show structure rather than noise, i.e. the remaining error is systematic. In the failure case with the largest error (the nine-input, high-contrast composite-conduction problem) the error is spread over the whole field, so no member of the pool captures this map at 256 runs; this is consistent with the benchmark, where networks beat the reduced-basis group on this problem.

6.6. Comparative analysis and real-data check

Comparison with the benchmark.. Taken together with section 6.1: on smooth problems with few inputs a Gaussian-process or well-regularised kernel member of the pool is hard to beat; on problems where the POD basis limits accuracy the full-field Gaussian process is the better choice; and where neural networks from the benchmark win (the composite-conduction problem with nine inputs), the default pool cannot match them (H1c). The tool's contribution is to find these cases from the training runs alone.

Real measured data.. Two real datasets were used (`data_real/`, CC BY 4.0): wind-tunnel measurements of airfoil noise (1,503 rows, five inputs) and the

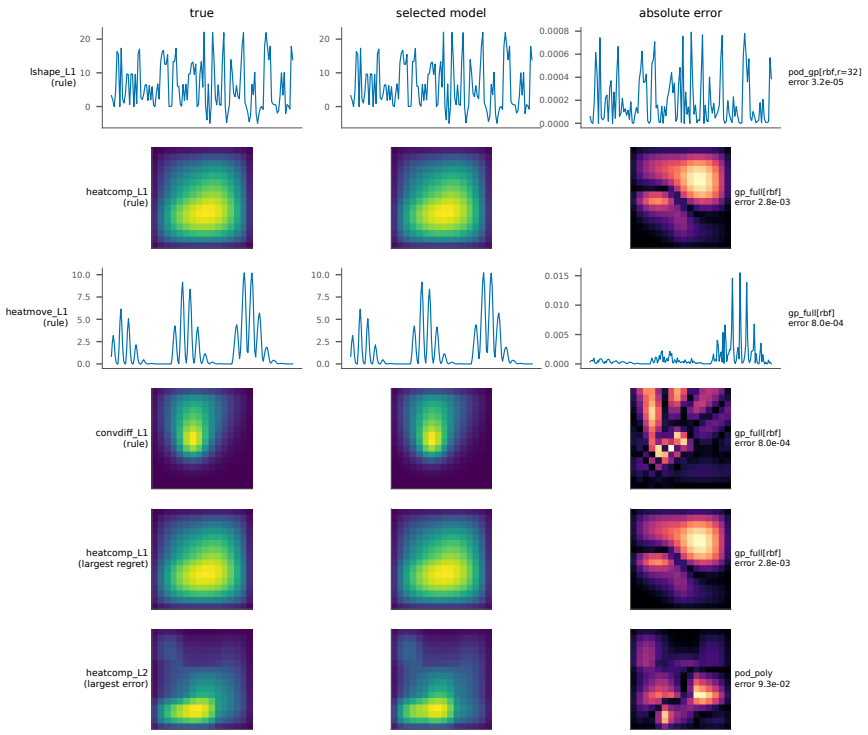


Figure 6: Selected-model predictions on single test samples chosen by a fixed rule (first test sample in dataset order; first problem of each family; plus the two failure cases defined in section 6.5). Columns: true field, selected model, absolute error.

output of a combined-cycle power plant (9,568 rows, four inputs). Both have a scalar, noisy output, so they violate assumption A2 and are not field data; the normalised RMSE replaces the relative error. With the protocol of Supplement addendum v3.1 (committed before any model was fitted), median regret was 1.04 over six cells (R1 met), the tool was no worse than always-POD+GP in 5 of 6 cells (R2 met), the tuned network was better by more than 25% in 0 cells (R3 met), and 9 of 12 forecasts were within a factor two (R4 met). We had predicted that R4 would fail because of measurement noise; it did not. The airfoil pool holds only 1,052 rows, so its $N = 1024$ results use nearly all of it. Table 7 lists the errors.

Plain-file validation and case study.. The three command-line commands were run on exported files for three problems; all sequences ran, and the error printed by `evaluate` matched an independent computation (Supplement section S7supplement.pdf). Figure 7 shows the unedited report for problem

Table 7: Real-data check: normalised RMSE (geometric mean over three seeds).

Dataset	N	Tool	Oracle	Always GP	Tuned net	Regret
airfoil	64	0.5960	0.5218	0.5868	0.6792	1.14
airfoil	256	0.3585	0.3578	0.3585	0.4072	1.00
airfoil	1024	0.2107	0.1845	0.2107	0.2037	1.14
ccpp	64	0.2867	0.2676	0.3153	0.5854	1.07
ccpp	256	0.2554	0.2539	0.2920	0.3682	1.01
ccpp	1024	0.2453	0.2437	0.2500	0.3530	1.01

```
(case study report pending)
```

Figure 7: Unedited SurroSelect report for `heatmove_L1` (256 runs, seed 0, target error 0.1%).

`heatmove_L1` with 256 runs and a target error of 0.1%.

7. Discussion

What the evidence supports.. On the independent problems, cross-validated selection from a small pool of classical models found the oracle or a near-oracle member (H1a), was not worse than always using a Gaussian process (H1b), flagged the cases in which the POD basis limited accuracy and a full-field model was better (H3), and gave usable forecasts of the error at four times the budget when starting from 64 runs (H2). Against a network tuned with twice the budget of any other method, the tool was met on the criterion of H4. On real measured data with scalar, noisy outputs, all four descriptive criteria were met.

What it does not support.. The benchmark finding that the reduced-basis group wins at large budgets did *not* hold on five of the eight independent problems (H5 not met), so a fixed recommendation "use a reduced-basis Gaussian process" is not safe; the case for the tool is that it finds out from the data. The tool did not match the benchmark's own selection among five methods that include networks (H1c not met): where networks win, the default pool, which has none, loses. The forecast was too optimistic from 256 runs, and its range is not an uncertainty interval. We did not isolate marginal-likelihood fitting as the mechanism behind the Gaussian-process advantage, and the pilot diagnostic that we had hoped would predict the winner from 32 runs was not confirmed (Supplement, section S3supplement.pdf).

Limitations..

1. **Simulated problems.** All 44 problems are simulated; the eight independent problems come from four families (two levels each) of one finite-element library. The real-data check uses two datasets with scalar, noisy outputs, which violate assumption A2; it says that the tool behaves sensibly on them, not that it works for noisy field data.
2. **Design knowledge.** The pool, the kernel-ridge grids and the criteria were designed with knowledge of the 36 earlier problems, so the independent problems are out-of-sample only in part. A numerical bug in the leave-one-out computation was found in a smoke test on one independent problem after some test errors had been seen and was fixed before the registered runs (deviation V9); the criteria were not changed.
3. **Few independent units.** Cells within a problem are not independent, so the intervals in table 4, which treat 24 cells as independent, are optimistic. Seeds share the pool and test set and nest their training subsets.
4. **Forecast.** The forecast is a power-law extrapolation without guarantee; it was too low by more than a factor two in 7 of 48 cases, and the pre-registered criterion was met with a confidence interval that includes the threshold.
5. **Networks.** The default pool contains no network. The tuned network uses 16 trials in a restricted space (depth 2–4, width 64–256) on one core; a more heavily tuned or different architecture could do better.
6. **Cost.** The solvers take milliseconds per run, so on these problems the tool costs more than the simulations it replaces; its use is justified for much slower simulations, which we did not test.
7. **Scope of the tool.** It assumes exchangeable runs on a common output grid, deterministic outputs, and a moderate number of inputs (only five of our 44 problems have ten or more).
8. **Process.** Hypotheses and code were committed before the runs they concern, but the history is our own, not an independent registry; several stages were designed after seeing earlier results (labelled post hoc); the independent reviews were performed by AI agents, not human reviewers.

Ethics and disclosure.. No human or animal subjects were involved. The two real datasets are public (CC BY 4.0). The AI involvement is disclosed in the acknowledgments.

8. Conclusion and Future Scope

We built SurroSelect, a procedure and open tool that chooses a stand-in for a slow simulation by cross-validation, reports ties, checks whether the POD basis limits accuracy and forecasts the error for more runs, and we tested it against criteria fixed in advance on eight independent finite-element problems. Selection was close to the oracle in all 24 cells and the basis check pointed to full-field models where they were better; the tool matched but did not beat the benchmark's selection among networks and Gaussian-process variants, and its forecast was usable from small budgets only. A 44-problem study showed that the winner depends on smoothness, the number of inputs, the budget, the training effort of the networks and how widely a kernel method's regularisation is searched, and that the simple rule "reduced basis wins at large budgets" fails on five of eight independent problems. Concrete next steps are (i) a pool that includes a tuned network, with a rule for when to spend its cost; (ii) a forecast that models the error floor and gives a calibrated interval; (iii) tests on real field data from simulations that are much slower than ours, with the full cost of the simulations included; (iv) a public benchmark such as PDEBench [32]; and (v) independent human replication.

Acknowledgments

Use of AI tools. The study design, code, analyses, figures and the drafting of this paper were carried out with an AI system (Claude, Anthropic) acting as a research assistant under the authors' direction. The internal "reviews" and audits mentioned in the supplement were performed by AI agents, not by human reviewers. The authors are responsible for the content, have checked it (to be confirmed by the authors before submission), and all authors must approve the submission. [Funding, other acknowledgments and competing-interest statement: to be supplied by the authors.]

Data and code availability

All data are generated by the code in the project repository from problem names and seeds; raw outputs, aggregation scripts (which write every number in this paper) and figure scripts are included, and `code/run_all.py` reruns everything. The tool is the `surroselect` package (MIT licence; copyright holder to be set by the authors). A permanent archive with a DOI has not been created yet: [authors to provide the public repository link or archive DOI before submission].

References

[1] J. S. Hesthaven, S. Ubbiali, Non-intrusive reduced order modeling of nonlinear problems using neural networks, Journal of Computational Physics 363 (2018) 55–78. doi:10.1016/j.jcp.2018.02.037.

[2] K. Bhattacharya, B. Hosseini, N. B. Kovachki, A. M. Stuart, Model reduction and neural networks for parametric PDEs, SMAI Journal of Computational Mathematics 7 (2021) 121–157. doi:10.5802/smai-jcm.74.

[3] M. Guo, J. S. Hesthaven, Reduced order modeling for nonlinear structural analysis using Gaussian process regression, Computer Methods in Applied Mechanics and Engineering 341 (2018) 807–826. doi:10.1016/j.cma.2018.07.017.

[4] S. Fresca, A. Manzoni, POD-DL-ROM: enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition, Computer Methods in Applied Mechanics and Engineering 388 (2022) 114181. doi:10.1016/j.cma.2021.114181.

[5] N. R. Franco, A. Manzoni, P. Zunino, A deep learning approach to reduced order modelling of parameter dependent partial differential equations, Mathematics of Computation 92 (340) (2023) 483–524. doi:10.1090/mcom/3781.

[6] N. McGreivy, A. Hakim, Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations, Nature Machine Intelligence 6 (10) (2024) 1256–1269. doi:10.1038/s42256-024-00897-5.

[7] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nature Machine Intelligence 3 (2021) 218–229. doi:10.1038/s42256-021-00302-5.

[8] Z. Li, N. Kovachki, K. Azizzadenesheli, et al., Fourier neural operator for parametric partial differential equations, in: International Conference on Learning Representations, 2021, arXiv:2010.08895.

[9] P. Batlle, M. Darcy, B. Hosseini, H. Owhadi, Kernel methods are competitive for operator learning, Journal of Computational Physics 496 (2024) 112549. doi:10.1016/j.jcp.2023.112549.

[10] J. Westermann, B. Huber, T. O'Leary-Roseberry, J. Zech, Performance of neural and polynomial operator surrogates, arXiv:2604.00689 (preprint) (2026).

[11] A. Bērziņš, J. Helmig, F. Key, S. Elgeti, Standardized non-intrusive reduced order modeling using different regression models with application to complex flow problems, arXiv:2006.13706 (preprint) (2020).

[12] C. Mora, A. Yousefpour, S. Hosseinmardi, H. Owhadi, R. Bostanabad, Operator learning with Gaussian processes, Computer Methods in Applied Mechanics and Engineering 434 (2025) 117581. doi:10.1016/j.cma.2024.117581.

[13] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: learning maps between function spaces with applications to PDEs, Journal of Machine Learning Research 24 (89) (2023) 1–97.

[14] K. Lee, K. T. Carlberg, Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders, Journal of Computational Physics 404 (2020) 108973. doi:10.1016/j.jcp.2019.108973.

[15] A. Cohen, R. DeVore, Approximation of high-dimensional parametric PDEs, Acta Numerica 24 (2015) 1–159. doi:10.1017/s0962492915000033.

[16] C. Greif, K. Urban, Decay of the Kolmogorov N-width for wave problems, Applied Mathematics Letters 96 (2019) 216–222. doi:10.1016/j.aml.2019.05.013.

[17] C. E. Rasmussen, C. K. I. Williams, Gaussian Processes for Machine Learning, MIT Press, 2006. doi:10.7551/mitpress/3206.001.0001.

[18] K. Duraisamy, Predictivity and utility of neural surrogates of multiscale PDEs, arXiv:2604.20061 (preprint) (2026).

[19] M. V. de Hoop, D. Z. Huang, E. Qian, A. M. Stuart, The cost-accuracy trade-off in operator learning with neural networks, Journal of Machine Learning 1 (3) (2022) 299–341. doi:10.4208/jml.220509.

[20] P. M. Wyder, J. Goldfeder, A. Yermakov, Y. Zhao, et al., Common task framework for a critical evaluation of scientific machine learning algorithms, arXiv:2510.23166 (preprint) (2025).

[21] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, Journal of Machine Learning Research 13 (2012) 281–305.

[22] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, A. Talwalkar, Hyperband: a novel bandit-based approach to hyperparameter optimization, Journal of Machine Learning Research 18 (2018) 1–52.

[23] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: a next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019. doi:10.1145/3292500.3330701.

[24] J. Dodge, S. Gururangan, D. Card, R. Schwartz, N. A. Smith, Show your work: improved reporting of experimental results, in: Proceedings of EMNLP-IJCNLP 2019, 2019, pp. 2185–2194. doi:10.18653/v1/d19-1224.

[25] X. Bouthillier, P. Delaunay, M. Bronzi, et al., Accounting for variance in machine learning benchmarks, in: Proceedings of Machine Learning and Systems (MLSys), 2021, arXiv:2103.03098.

[26] B. Williams, S. Cremaschi, Surrogate model selection for design space approximation and surrogate-based optimization, Computer Aided Chemical Engineering 47 (2019) 353–358. doi:10.1016/B978-0-12-818597-1.50056-4.

[27] F. Mohr, J. N. van Rijn, Learning curves for decision making in supervised machine learning: a survey, Machine Learning 113 (2024) 8371–8425. doi:10.1007/s10994-024-06619-7.

[28] J. S. Rosenfeld, A. Rosenfeld, Y. Belinkov, N. Shavit, A constructive prediction of the generalization error across scales, in: International Conference on Learning Representations, 2020, arXiv:1909.12673.

[29] R. Milk, S. Rave, F. Schindler, pyMOR – generic algorithms and interfaces for model order reduction, SIAM Journal on Scientific Computing 38 (5) (2016) S194–S216. doi:10.1137/15M1026614.

[30] S. Marelli, B. Sudret, UQLab: a framework for uncertainty quantification in Matlab, in: Vulnerability, Uncertainty, and Risk (ICVRAM 2014), 2014. doi:10.1061/9780784413609.257.

[31] T. Gustafsson, G. D. McBain, scikit-fem: a Python package for finite element assembly, Journal of Open Source Software 5 (52) (2020) 2369. doi:10.21105/joss.02369.

[32] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, M. Niepert, PDEBench: an extensive benchmark for scientific machine learning, neurIPS 2022 Datasets and Benchmarks Track; arXiv:2210.07182 (2022).

[33] G. H. Golub, M. Heath, G. Wahba, Generalized cross-validation as a method for choosing a good ridge parameter, Technometrics 21 (2) (1979) 215–223. doi:10.1080/00401706.1979.10489751.

[34] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212. doi:10.1080/01621459.1927.10502953.

[35] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.